

031 练习详解与故障报告

本答案把计算结果、定位判断和结论边界分开。数值对应本讲固定CPU实验，尾数受运行版本与平台影响。逻辑判断不应依赖浮点极限处的最后几位。

1 两条样本的梯度、范数和更新

给定 $x = (-1, 1)$ 、 $y = (-1, 1)$ 、 $w = 1/2, b = 1/4$ ，预测为 $(-1/4, 3/4)$ ，残差为 $(3/4, -1/4)$ 。half-MSE为 $[(3/4)^2 + (-1/4)^2]/4 = 5/32$ 。损失的归约按 $n = 2$ 平均：

$$g_w = [(3/4)(-1) + (-1/4)(1)]/2 = -1/2,$$

$$g_b = [3/4 - 1/4]/2 = 1/4, \quad \|g\|_2 = \sqrt{5}/4.$$

学习率 $\eta = 1/10$ 的SGD得到 $w' = 1/2 + 1/20 = 11/20$ 、 $b' = 1/4 - 1/40 = 9/40$ 。新残差为 $27/40, -9/40$ ；损失 $[(27/40)^2 + (-9/40)^2]/4 = 81/640$ 。更新范数为 $\eta\|g\|_2 = \sqrt{5}/40$ ，它等于实际差向量 $(1/20, -1/40)$ 的范数。

若把half-MSE改为半平方和，损失与梯度都变为原来的2倍。在同一个参数点、同一学习率下，一步SGD更新也放大2倍。本讲4样本探针则放大4倍。这不是“sum一定错误”，而是你改变了目标尺度；若意图仍是原来的均值，就发生了契约违背。

广播题中，正确残差是 2×1 的全零矩阵。标签压成长度2向量后，广播产生 $\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$ ，四个元素平方均值为 $1/2$ ，再乘 $1/2$ 得到 $1/4$ 。应该在减法前验证shape，而不是在结果后靠loss是否“正常”判断。

2 同一13参数例的链式累计答案

先计算正文3.1节的A、H、预测、残差，再把残差除以4得到R。第一层权重W1[1,1]的四条路径是R乘输出权重0.3，再乘tanh局部导数，最后乘第一输入坐标。贡献分别约为0.000950632、-0.020343867、-0.043531562、0，和为-0.062924797。第一层偏置b1[1]最后一段局部导数恒为1，四项为-0.000950632、0.020343867、0.043531562、-0.034650466，和为0.028274331。

输出权重W2[1,1]不经过tanh反向；它的每条路径直接是隐藏输出H的第一列乘R，和约为-0.010982235。输出偏置b2每条局部导数为1，直接把四个R相加，得0.095656901。四种参数的公式不同，不能统一写成“误差乘输入”而不说明误差位于哪一层。

完整13维梯度按向量索引顺序为：

```
[ -0.062924797,  0.073144405, -0.036280542,
  0.079132659, -0.090250674,  0.045125337,
  0.028274331, -0.032068797,  0.015742738,
 -0.010982235,  0.114178445, -0.117138766,
  0.095656901]
```

学习率0.1时， $W1[1,1]$ 从0.2变为约0.206292480， $b1[1]$ 从0.05变为约0.047172567；全部新参数及4行新A、H、预测见正文完整数值表，并在full-trace.json中保留全精度。新预测约为(-0.124434567,-0.245317786,-0.350856871,0.139195921)，新loss为0.07241983164894575。必须重新计算隐藏层，不能只把新末层参数乘旧H。

新s0误差绝对值从约0.013484增加到0.024435，但批次总损失仍下降。原因是4个样本共享同一参数；SGD降低的是平均目标的一阶近似，不能保证每条样本都单调改善。

3 中央差分、误差与容差

沿一个参数坐标把函数记为 $\phi(t) = L(\theta + te_k)$ 。在足够平滑时， $\phi(h) = \phi(0) + h\phi'(0) + h^2\phi''(0)/2 + h^3\phi'''(0)/6 + o(h^3)$ ； $\phi(-h)$ 中奇次项变号。相减除以 $2h$ 后，得到 $\phi'(0) + h^2\phi'''(0)/6 + o(h^2)$ ，故主截断误差是二阶。

若一次函数求值误差的量级是 uC ，两个近似值相减再除以 $2h$ 可能产生约 uC/h 的误差。缩小 h 先降低截断误差，继续缩小则可能放大舍入误差。扫描多个步长并检查稳定区间，比固定相信一个极小数可靠。二次手算例三阶导数为0，但浮点相减误差仍存在。

本例 $h = 10^{-6}$ 时13坐标最大绝对误差约 1.2061×10^{-11} 。通过标准是 $|a - b| \leq 10^{-8} + 10^{-5} \max(|a|, |b|)$ ；例如 $a = 0, b = 10^{-10}$ 可通过绝对容差，不需要计算无定义的相对误差。容差应先有数值依据，不能在失败后任意放大直到绿灯。

方向题的正确梯度为(1, 2)、错误梯度为(2, 1)，用单位方向 $v = (1, 1)/\sqrt{2}$ ，两者内积都为 $3/\sqrt{2}$ 。误差向量(1, -1)恰好与 v 正交。检查更多独立方向可以降低漏检风险；逐坐标检查才覆盖这里全部坐标。有限数量与有限输入点仍不等于全域形式证明。

4 为什么0-8号失败直接指向第一层

展平映射是：0-5号为 W_1 ，6-8号为 b_1 ，9-11号为 W_2 ，12号为 b_2 。正常与分离隐藏层的前向loss都为0.07877855276232809，说明当前数值相同；差分扰动第一层参数仍会重新计算隐藏输出，所以数值导数非零。

`h.detach()`令反向停止经过隐藏层。对于作为一个叶向量的13维参数，autograd仍会因末层参数而返回整个向量，但前9个坐标为0。换成分别登记的`nn.Parameter`， $W1$ 和 $b1$ 在清空后会保持`grad is None`。这个差别来自参数组织和图连接方式，不是两个实验互相矛盾。

本例0-8号全部失败，最大差约0.090250674；末层仍通过。优先排查第一层输出到第二层输入之间的detach、重新构造张量、转入无求导历史的外部库或错误的no_grad范围。应先找第一处丢失路径的边，不必重写全部网络。

标签反转却能通过gradcheck，因为正反两种计算都在求同一个“错误配对目标”的导数。应通过ID、原始标签含义和配对检查发现它，而不是期待导数检查理解任务。

5 ReLU、None与零更新

ReLU在0左右的差商分别趋向0和1，普通导数不存在。中央差分为 $[h - 0]/(2h) = 1/2$ ，PyTorch在这个点的反向约定返回0。不能把0.5当唯一“正确答案”。移到 $x = -1$ 时局部函数恒为0，普通导数就是0；此时返回的是数值0张量，而不是没有连接的None。

完成清空、前向和反向之后，本应参与的叶参数仍为None，检查其requires_grad、是否在前向被使用以及图是否被切断。非叶中间张量的.grad默认不保存，须先区分对象角色。数值0表示导数已计算但在当前点为零，可能由饱和、平坦区、残差为零或不同路径抵消造成。

非零梯度却零更新，应查step()是否执行、学习率是否为0、优化器是否持有那个同一对象以及冻结规则。不能仅凭梯度列表存在就假定优化器管理了它。

探针	W1梯度	W1更新	关键额外证据
正常SGD	0.470573	0.0141172	lr=0.03, W1在优化器中
no_step	0.470573	0	未执行step
zero_lr	0.470573	0	执行step, 但lr=0
omit_W1	0.470573	0	W1不在参数列表, 其他组更新
detach_hidden	None	0	W1、b1都无梯度, 末层正常
sum_reduction	1.88229	0.0564687	当前批次4条, 损失也放大4倍

表中的梯度和更新均为二范数。以上更新比例只适用于本探针的无动量SGD；Adam及有历史状态的优化器要检查实际更新，不能套同一等式。

6 能拟合与不能拟合各说明什么

规则标签未损失约 3.71×10^{-33} ，固定随机标签未损失约 1.76×10^{-22} ，均远低于 10^{-4} 的诊断阈值。结论是固定49参数网络与当前初始化、Adam设置及1600步预算能满足这8条约束。不能推断随机标签含可迁移规律，更不能推断某个真实任务准确率会高。

本实验没有训练外评价；训练点误差再低也不能填补这个证据空白。随机标签测试若失败，也可能只是容量、优化难度或预算限制，应继续缩小故障范围，而不是自动认定框架错误。

矛盾重复标签的输出必须相同，记为 c ，于是 $L(c) = [(c+1)^2 + (c-1)^2]/4 = (c^2 + 1)/2 \geq 1/2$ 。最小值在 $c = 0$ 取得，导数为 c 。实际300步到达约0.5与解析解一致。增加确定性网络宽度不能突破这一目标下界。

7 输入扰动与误差尺度

输入与标签一起反转，仅改变求和顺序，损失不变。本例原始与联合置换loss都约 3.71×10^{-33} 。只反转输入时配对被破坏，loss约0.975；全部输入置零后约0.243756，接近标签均值预测器的0.24375。这些检查说明当前程序对所选输入变化有响应，但不能唯一识别模型使用了怎样的特征机制。

小扰动后的变化率分别约0.58414、0.58387、0.58383；生成规则为0.7。网络只被8个训练点的函数值约束，而没有被要求在这些点附近具有梯度0.7，所以原点误差极小不保证邻域斜率正确。

若单样本局部斜率为 a_i ，预测对原点已拟合，则扰动后的目标与预测差约 $(a_i - 0.7)\epsilon$ 。half-MSE约为 $\epsilon^2 \sum_i (a_i - 0.7)^2 / (2n)$ 。因此缩小 ϵ 使绝对loss按平方缩小，可以与斜率误差不消失同时发生。应同时看变化率或归一化误差，且不将有限点的平均率等同于整个分布的鲁棒性。

8 参考故障报告：优化器漏传W1

预期。 在前4条固定合成样本上，Tiny的W1、b1、W2、b2应都参与无动量SGD，学习率0.03。从相同seed31初始化出发，一步参数变化应与负梯度方向对应。

最小复现。 Python3.12、PyTorch2.7.1 CPU、float64；调用`one_step_probe(X[:4], y[:4], "omit_w1")`。X为(4,2)，y为(4,1)，loss是half-MSE。清空整个模型梯度，计算一次前向、一次反向和一次优化器step。

观察。 loss为0.3618884807948939。W1的梯度范数为0.47057259402887297，更新范数为0；其优化器成员身份为False。b1、W2、b2均在优化器内，并发生非零更新。loss与正常探针一致，所有梯度也与正常探针一致。

定位。 证据排除了“W1反向路径被完全切断”和“整个step没有执行”作为本例的解释。检查列表构造发现过滤掉了名称为W1的参数。根因位于优化器参数选择，不位于损失归约或模型容量。

单变量修复。 在同一初始状态重新构造优化器，传入全部`model.parameters()`；其余条件不变。W1更新范数变为0.01411717782086618，约等于 $0.03 \times 0.47057259402887297$ ，另外三组仍按原来方式更新。

回归检查。 验证W1对象出现在优化器中；对所有参数组检查梯度非None且有限；用`assertAlmostEqual(update_norm, 0.03*grad_norm, places=12)`核对本SGD探针。检查必须在step后和下一次清梯度前进行。生产输入验证使用显式异常，不能依赖可被python -O移除的普通assert。

范围。 本报告只证明固定条件下的漏参故障已修复，没有评价真实数据、其他精度、复杂优化器或泛化。若存在第二个故障，仍需后续检查。

9 另一份报告应怎样不同：隐藏层分离

使用`detach_hidden`时，`loss`与正常探针相同，`W1`与`b1`的梯度均为`None`，二者更新均为0；`W2`、`b2`仍有梯度且更新正常。此时优化器确实持有全部参数。因此“把`W1`加入优化器”不是有效修复。

最小修复是移除非预期的隐藏输出分离，恢复从输出损失回到第一层的路径；重建相同初始状态，再核对`W1`、`b1`获得有限梯度，以及13参数例的0-8号差分重新通过。还应保留负例测试：故意重新加入分离时，测试应失败或明确记录预期失败，否则回归测试可能根本没覆盖这条路径。

如果分离是算法定义的一部分，就不应擅自移除，而应在目标说明中注明停梯度意图。本讲将它标为故障，是因为约定的完整`tanh`网络要求第一层参与同一损失的学习。